- Deployment & Usage
  - Prerequisites
  - Installation
    - 1. Create Virtual Environment
    - 2. Install Dependencies
    - Key Dependencies
    - 3. Configure Environment
  - Running the System
    - Terminal 1: Start the MCP Server
    - Terminal 2: Run Analysis
    - Output Location
  - Configuring Analysis Requests
    - Example Requests
    - Language Support
  - REST API (Future UI Integration)
    - Endpoints
    - Submit Analysis
    - Check Status
  - Tuning & Configuration
    - LLM Model
    - Context Window Controls
    - Pipeline Controls
    - Downtime Threshold
  - Troubleshooting
    - Data Loader Returns Empty
    - Analyst Doesn't Call execute_python
    - Reporter Calls query_mill_data
    - Recursion Limit Error
    - MCP Connection Refused
    - Empty Output Directory
  - Architecture Diagram (Deployment View)
    - Process Isolation

Deployment & Usage

Prerequisites

- **Python 3.11+** (tested with 3.11)
- **PostgreSQL** access to the `em_pulse_data` database
- **Google API Key** for Gemini LLM access

Installation

1. Create Virtual Environment

```
python -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate     # Linux/macOS
```

2. Install Dependencies

```
cd python/agentic
pip install -r requirements.txt
```

Key Dependencies

Package	Purpose
`mcp`	Model Context Protocol server & client
`langgraph`	Multi-agent state graph orchestration
`langchain-core`	Base message types, tool abstractions
`langchain-google-genai`	Google Gemini LLM integration
`pandas`, `numpy`	Data analysis
`matplotlib`, `seaborn`	Chart generation

Package	Purpose
`scipy`, `scikit-learn`	Statistical analysis
`sqlalchemy`, `psycopg2-binary`	PostgreSQL connectivity
`starlette`, `uvicorn`	MCP server HTTP transport
`python-dotenv`	Environment variable management
`pydantic`	Schema validation for tool arguments
`fastapi`	REST API endpoint (future UI)

3. Configure Environment

Create `python/agentic/.env`:

```
GOOGLE_API_KEY=your_google_api_key_here

DB_HOST=em-m-db4.ellatzite-med.com
DB_PORT=5432
DB_NAME=em_pulse_data
DB_USER=your_username
DB_PASSWORD=your_password
```

> **Security:** Never commit `.env` to version control. The `.env.example` file shows the expected format.

Running the System

The system requires **two processes** running simultaneously:

Terminal 1: Start the MCP Server

```
cd python/agentic
python server.py
```

Expected output:

```
🏭 Agentic Data Analysis MCP Server starting...
   Tools registered: ['get_db_schema', 'query_mill_data', 'query_combined_data',
                      'execute_python', 'list_output_files',
'write_markdown_report']
INFO:     Uvicorn running on http://0.0.0.0:8003 (Press CTRL+C to quit)
🚀 Server is running on http://localhost:8003/mcp
```

Terminal 2: Run Analysis

```
cd python/agentic
python main.py
```

Expected output flow:

```
Connecting to MCP server at http://localhost:8003/mcp...
Connected to MCP server at http://localhost:8003/mcp

Tools loaded from MCP server:
  - get_db_schema: Return the database schema metadata...
  - query_mill_data: Load time-series process data...
  - query_combined_data: Load mill process data + ore quality...
  - execute_python: Execute Python code for data analysis...
  - list_output_files: List all files currently in the output/...
  - write_markdown_report: Write a Markdown report file...

═══════════════════════════════════════════════════════════════════
  🏭 ANALYSIS REQUEST: Mill Ore Load Comparison
  Сравни средното натоварване по руда на всички мелници...
═══════════════════════════════════════════════════════════════════

  [data_loader] iteration 1/5 — processing...
  [data_loader] Calling tools: ['query_mill_data', 'query_mill_data', ...]
    [tool] Executing query_mill_data...
    ...

  [analyst] iteration 1/5 — processing...
  [analyst] Calling tools: ['execute_python']
    [tool] Executing execute_python...
    ...

  [reporter] iteration 2/5 — processing...
  [reporter] Calling tools: ['write_markdown_report']
    ...

  ──→ Pipeline complete!

  ✅ Analysis complete. Check agentic/output/ for results.
```

Output Location

All generated files are saved to `python/agentic/output/`:

```
output/
├── ore_comparison.png           # Bar chart comparing mills
├── ore_std_comparison.png       # Standard deviation comparison
├── ore_histograms.png           # Distribution histograms
└── mill_comparison_report.md    # Final Markdown report
```

Configuring Analysis Requests

Edit `DEMO_REQUESTS` in `main.py`:

```
DEMO_REQUESTS = [
    (
        "Label for the analysis",          # Display label
        "Natural language analysis request" # Sent to the agents
    ),
]
```

Example Requests

Multi-mill comparison (Bulgarian):

```
(
    "Mill Ore Load Comparison",
    "Сравни средното натоварване по руда на всички мелници за последните 72 часа. "
    "Генерирай сравнителни графики. Изчисли също стандартните отклонения, "
    "сравни ги графично и дай хистограмите. "
    "При изчисленията елиминирай престоите (там където рудата е по-малка от 50 т/
ч). "
    "Мелница 11 по принцип работи с ниско натоварване.",
),
```

Single-mill comprehensive analysis (English):

```
(
    "Mill 8 Comprehensive Analysis",
    "Perform a comprehensive analysis of Mill 8 for the last 30 days. "
    "Include: EDA with distribution plots for key variables (Ore, PSI80, DensityHC,
MotorAmp), "
    "SPC control charts for PSI80 and Ore feed rate, "
    "correlation heatmap between all numeric variables, "
    "anomaly detection using Z-scores for PSI80 and Ore, "
    "and a downtime analysis identifying periods where Ore < 10 t/h. "
    "Generate a full report with charts and recommendations for the plant
manager.",
),
```

Language Support

The system works with requests in **both Bulgarian and English**. The Gemini LLM handles multilingual input natively. The domain context in prompts is in English, but the LLM adapts its output language to match the request.

REST API (Future UI Integration)

The `api_endpoint.py` file provides a FastAPI router for integrating with a web UI:

Endpoints

Method	Path	Description
`POST`	`/api/v1/agentic/analyze`	Submit an analysis request
`GET`	`/api/v1/agentic/status/{id}`	Check analysis progress
`GET`	`/api/v1/agentic/reports`	List all generated files
`GET`	`/api/v1/agentic/reports/{filename}`	Download a specific file

Submit Analysis

```
curl -X POST http://localhost:8000/api/v1/agentic/analyze \
  -H "Content-Type: application/json" \
```

```
  -d '{
    "question": "Analyze Mill 8 for the last week",
    "mill_number": 8,
    "start_date": "2026-03-04",
    "end_date": "2026-03-11"
  }'
```

Response:

```
{
  "analysis_id": "a1b2c3d4",
  "status": "running",
  "message": "Analysis started. Use GET /status/a1b2c3d4 to check progress.",
  "started_at": "2026-03-11T15:30:00"
}
```

Check Status

```
curl http://localhost:8000/api/v1/agentic/status/a1b2c3d4
```

Response:

```
{
  "analysis_id": "a1b2c3d4",
  "status": "completed",
  "question": "Analyze Mill 8 for the last week",
  "final_answer": "ACCEPT: Report written successfully.",
  "report_files": ["mill_8_analysis.md"],
  "chart_files": ["ore_distribution.png", "psi80_spc.png", ...],
  "started_at": "2026-03-11T15:30:00",
  "completed_at": "2026-03-11T15:32:45"
}
```

> **Note:** The API endpoint currently references `graph.py` (v1). To use the v2 pipeline, update the import in `api_endpoint.py` from `from graph import build_graph` to `from graph_v2 import build_graph`.

Tuning & Configuration

LLM Model

```
# graph_v2.py
GEMINI_MODEL = "gemini-3.1-flash-lite-preview"
```

Change this to use a different Gemini model. Larger models (e.g. `gemini-2.0-flash`) may produce better tool calls but cost more and are slower.

Context Window Controls

```
MAX_TOOL_OUTPUT_CHARS = 2000   # Truncate tool results sent to LLM
MAX_AI_MSG_CHARS = 3000        # Truncate AI message history
MAX_MESSAGES_WINDOW = 14       # Keep last N messages in context
```

Increase these for more detailed context at the cost of token usage. Decrease if the LLM is producing empty or confused responses (context overflow).

Pipeline Controls

```
MAX_SPECIALIST_ITERS = 5       # Max tool-call loops per specialist
MAX_REWORKS_PER_STAGE = 1      # Manager can rework each stage once
recursion_limit = 100          # LangGraph total node visits
```

- `MAX_SPECIALIST_ITERS`: Increase if the analyst needs more iterations for complex analyses
- `MAX_REWORKS_PER_STAGE`: Increase for stricter quality control (but watch for infinite loops)
- `recursion_limit`: Must be high enough to accommodate all stages × iterations × reworks

Downtime Threshold

The current prompt defines downtime as `Ore < 10 t/h` for single-mill analysis and `Ore < 50 t/h` for operational filtering. Adjust these values in the user request or in the

prompts.

Troubleshooting

Data Loader Returns Empty

Symptom: `[data_loader] Done: ""` — no tool calls made.

Causes:

1. Context too large — prior messages flooding the LLM
2. Tool binding failed — data_loader doesn't see `query_mill_data`
3. Gemini returns content as list instead of string

Fix: Check the debug output for `Context:` and `Raw response:` lines. Ensure `build_focused_context` returns the user request as the first message.

Analyst Doesn't Call execute_python

Symptom: Analyst returns text instead of tool calls.

Causes:

1. Too many data_loader tool results in context
2. Per-specialist tool binding not working
3. Gemini confusion from mixed-language prompts

Fix: Verify per-specialist tool binding is active. Check that `build_focused_context` properly filters prior-stage messages.

Reporter Calls query_mill_data

Symptom: Reporter tries to load data instead of writing a report.

Causes: Reporter has access to data-loading tools (missing per-specialist binding).

Fix: Ensure `TOOL_SETS["reporter"]` only contains `["list_output_files", "write_markdown_report"]`.

Recursion Limit Error

Symptom: `GraphRecursionError` during execution.

Fix: Increase `recursion_limit` in `main.py`:

```
config={"recursion_limit": 150}
```

MCP Connection Refused

Symptom: `ConnectionRefusedError` when running `main.py`.

Fix: Ensure the MCP server is running on port 8003 (`python server.py`) before starting `main.py`.

Empty Output Directory

Symptom: No `.png` or `.md` files in `output/`.

Causes:

1. The analyst's Python code errored (check `execute_python` result for `"error"` key)
2. Charts were saved to wrong directory
3. The pipeline didn't reach the analyst/reporter stages

Fix: Check the full pipeline output for error messages. Verify `OUTPUT_DIR` points to the correct path.

Architecture Diagram (Deployment View)

```
┌──────────────────────────────┐
│  Terminal 1: MCP Server      │
│  python server.py            │
│  Port: 8003                  │
│                              │
│   ┌────────────────────────┐ │
│   │ Starlette + uvicorn    │ │
│   │ StreamableHTTP MCP     │ │
│   ├────────────────────────┤ │
│   │ Tool Registry          │ │
│   │ • db_tools (3 tools)   │ │
│   │ • python_executor      │ │
│   │ • report_tools         │ │
│   ├────────────────────────┤ │
│   │ In-Memory Store        │ │
│   │ _dataframes dict       │ │
│   └────────────────────────┘ │
│            │                 │
│            ▼                 │
│   ┌────────────────────────┐ │
│   │ PostgreSQL             │ │
│   │ em_pulse_data          │ │
│   └────────────────────────┘ │
└──────────────────────────────┘

┌──────────────────────────────┐
│  Terminal 2: LangGraph       │
│  python main.py              │
│                              │
│   ┌────────────────────────┐ │
│   │ MCP Client Session     │ ──────► HTTP to :8003
│   ├────────────────────────┤ │
│   │ LangGraph Pipeline     │ │
│   │ • data_loader          │ │
│   │ • analyst              │ │
│   │ • code_reviewer        │ │
│   │ • reporter             │ │
│   │ • manager (QA)         │ │
│   ├────────────────────────┤ │
│   │ Gemini LLM             │ ──────► Google AI API
│   └────────────────────────┘ │
└──────────────────────────────┘
```

Process Isolation

The MCP server and the LangGraph client run in **separate Python processes**. This means:

- The server can be restarted without affecting the client (and vice versa)
- DataFrames live in the **server's** memory — the client never sees raw data

- Tool execution happens server-side — the LLM only sees tool results
- Multiple clients could connect to the same server simultaneously